dfnkdfnsfdflmdslx cxlnfd

sdnkfnkejofpsldf;ksasaf

🎨 Excalidraw Whiteboard + PDF Export & GitHub Backup

Complete A to Z Codebase & Architecture Guide (Explained in Simple Hinglish for a 10-Year-Old)

🧸 CHAPTER 1: Yeh Project Kya Hai? (Real-Life Kahani)

Imagine karo aapke paas ek **Jadui Whiteboard (Magic Slate)** hai jahan aap kuch bhi kar sakte ho:

- Pencil se notes likh sakte ho, diagrams bana sakte ho, maths ke formulas solve kar sakte ho.
- Rectangles, arrows, circles bana sakte ho, aur Google se koi bhi photo copy karke direct paste kar sakte ho.

Lekin is magic board par humne **2 Special Superpowers (Buttons)** lagayi hain:

1. "Download PDF" (Magic Photocopy Machine 🖨️):
Jab aapki padhai poori ho jaye, aap ye button dabate ho. Ye turant board par jo kuch bhi hai, bina ek bhi line kate, crystal-clear high-definition **PDF document** bana kar aapke computer me download kar deta hai!

2. "Push to GitHub" (Personal Courier Boy 🐙):
Maan lo aaj aapne **Science ka "Day 1"** padha. Aap chahte ho ki ye notes aapke online digital locker (GitHub repository) me safely jama ho jayein. Aap "Push to GitHub" dabate ho, file ka naam `Day-1.pdf` rakhte ho, aur ye seedha aapke GitHub repo me chala jata hai! Jab Science ka syllabus khatam ho jaye, toh 1-click me repo **Unlink** karke **Maths** ka repo connect kar sakte ho.

🧩 CHAPTER 2: In-Built vs Custom (Kaun Si Cheez Readymade Hai aur Kaun Si Humne Banayi?)

Coding me do tarah ke hisse hote hain: Jo bazaar se readymade mila (In-built) aur jo humne hath se code likha (Custom).

Category	Cheez Ka Naam	Asal Me Yeh Kya Karta Hai?
IN-BUILT	`@excalidraw/excalidraw`	Main whiteboard drawing engine (pencil, shapes, eraser, colors, zoom, hand-drawn look).
IN-BUILT	`jspdf`	PDF generate karne ka tool jo kisi image/drawing ko PDF sheet me fit karta hai.
IN-BUILT	`lucide-react`	Icons (Download icon, File icon, GitHub icon, Settings gear).
IN-BUILT	`Vite + React`	Website ko browser me super-fast chalane wala framework.
CUSTOM	`src/utils/pdfExport.js`	Hamara custom PDF generator jo whiteboard drawing ko 2x high-resolution me crop karke PDF banata hai.
CUSTOM	`src/utils/githubSync.js`	Hamara custom GitHub cloud sync logic jo bina backend ke direct GitHub API se file upload karta hai.
CUSTOM	`GitHubConnectModal.jsx`	Popup window jahan repo link aur token enter ya Unlink kiya jata hai.
CUSTOM	`GitHubPushModal.jsx`	Popup window jahan `Day-1.pdf` naam likh kar direct repo me push kiya jata hai.
CUSTOM	`src/App.jsx & App.css`	Top-right me clean buttons lagana, auto-save sambhalna, aur elegant styling.

📂 CHAPTER 3: Har Ek File Aur Function Ka Post-Mortem

1. File: `src/utils/pdfExport.js` (PDF Generation Engine)

Is file ka kaam whiteboard ke sketches ko digital PDF me convert karna hai.

- **`generatePdfDoc(excalidrawAPI, options)`:** Scene ke saare elements nikaalta hai. Agar canvas khali hai toh error deta hai. `exportToCanvas()` call karke 2x Retina crisp image banata hai aur `jsPDF` ke through exact sheet par center karke fit karta hai.
- **`exportWhiteboardToPdf(excalidrawAPI, options)`:** `generatePdfDoc()` ko call karke browser me seedha file download (`doc.save`) karwata hai.
- **`exportWhiteboardPdfBlob(excalidrawAPI, options)`:** Computer me download karne ke bajaye PDF ka raw binary data (Blob) memory me return karta hai, taaki GitHub par upload kiya ja sake.

2. File: `src/utils/githubSync.js` (GitHub Sync Engine)

Is file ka kaam direct browser se GitHub API ke zariye files upload karna hai.

- **`getSavedGitHubConfig()`:** Browser ki memory (`localStorage`) se check karta hai ki koi repo link hai ya nahi.
- **`saveGitHubConfig(config)`:** Repo name, token, folder, aur branch ko `localStorage` me save karta hai taaki page refresh karne par dobara connect na karna pade.
- **`removeGitHubConfig()`:** LocalStorage se repo info delete karta hai (Unlink action) taaki aap naya subject connect kar sako.
- **`parseRepoString(input)`:** Smart helper hai — chahe aap `username/science-notes` likho ya poora URL paste karo, ye sahi se owner aur repo alag-alag pehchan leta hai.
- **`verifyGitHubAccess(owner, repo, token)`:** GitHub par test call karke check karta hai ki token aur repo valid hain ya nahi.
- **`blobToBase64(blob)`:** PDF binary file ko Base64 text string me badalta hai (kyunki GitHub API text form me data leti hai).
- **`uploadFileToGitHub(options)`:** Sabse main function! Ye GitHub par check karta hai ki kya same naam ki file pehle se maujood hai? Uska SHA hash nikaalta hai, aur `PUT /repos/{owner}/{repo}/contents/{path}` request bhej kar file commit kar deta hai.

3. File: `src/components/GitHubConnectModal.jsx`

Popup window jo **Link Repo** ya settings gear dabane par khulti hai:

- **`handleSave()`:** User ke dale hue token aur repo ko verify karke save karta hai.
- **`handleUnlink()`:** Current subject repo ko disconnect karta hai taaki naya subject link ho sake.

4. File: `src/components/GitHubPushModal.jsx`

Popup window jo **Push to GitHub** dabane par khulti hai:

- **`handlePush()`:** `exportWhiteboardPdfBlob()` se PDF banata hai, `uploadFileToGitHub()` se push karta hai, aur screen par green tick ke sath live GitHub link dikhata hai.

5. File: `src/App.jsx` (Central Controller)

Whiteboard ka main controlling hub:

- **`handleCanvasChange()` (Debounced Auto-Save):** Jaise hi aap canvas par kuch draw karte ho, 400 milliseconds ke andar wo automatically browser ke `localStorage` me save ho jata hai. Refresh karne par kuch bhi delete nahi hota!
- **`renderTopRightUI()`:** Canvas ke top-right corner me "Download PDF" aur "Push to GitHub" / "Link Repo" ke buttons render karta hai.

🔄 CHAPTER 4: Step-by-Step Data Flow (Kaise Kaam Hota Hai?)

Flow 1: Jab Aap Canvas Par Notes Likhte Ho

```
Aapne canvas par "Diagram" banaya
   |
   ▼
[handleCanvasChange] trigger hota hai (400ms debounce)
   |
   ▼
Browser ke localStorage me automatically JSON string save ho jati hai
```

Flow 2: Jab Aap "Download PDF" Click Karte Ho

```
User clicks "Download PDF"
   |
   ▼
[generatePdfDoc] ──> 2x High-DPI Canvas Image banti hai
   |
   ▼
[jsPDF] ──> Image ko exact PDF sheet par fit karta hai
   |
   ▼
Browser triggers download: "excalidraw-2026-10-25.pdf"
```

Flow 3: Jab Aap "Push to GitHub" Click Karte Ho (Day-1 Notes)

```
User types "Day-1.pdf" aur clicks "Push PDF to Repo"
   |
   ▼
[exportWhiteboardPdfBlob] ──> PDF Blob memory me banta hai
   |
   ▼
[blobToBase64] ──> Binary PDF Base64 text me convert hota hai
   |
   ▼
[uploadFileToGitHub] ──> PUT https://api.github.com/repos/{owner}/{repo}/contents/{path}
   |
   ▼
GitHub commits file ──> Screen par clickable "View on GitHub" button aa jata hai!
```

Flow 4: Naye Subject Par Switch Karna (Unlink & Link)

```
Aaj Science padha: "username/science-notes" link tha ──> Day-1.pdf push ho gaya!
Kal Maths padhna hai:
1. Click repo button "science-notes ⚙"
2. Click "Unlink Repo"
3. Enter "username/math-notes" aur Connect dabao
4. Ab Maths ka Day-1.pdf push karo! Simple and clean!
```


🔒 CHAPTER 5: Security & Privacy (Kya Mera Token Safe Hai?)

100% Safe & Private:

- **No External Server:** Koi third-party server nahi hai jahan aapka token ja sake.
- **Local Storage Only:** Token sirf aur sirf aapke computer ke browser me rehta hai.
- **Direct Encrypted HTTPS:** Browser direct official GitHub server (`api.github.com`) se secure SSL tunnel ke zariye baat karta hai.
- **Instant Revoke:** Aap GitHub settings me jakar kisi bhi waqt token delete ya revoke kar sakte ho.